Assignment 3: Database Design Analysis

1. Introduction

Database design analysis checks whether data is stored in a clear and reliable way. This assignment analyses the Hotel Management System, which stores guest details, rooms, bookings, payments, and optional services.

2. Tables in the proposed design

Table	Purpose	Primary key
Guests	Stores each guest's contact details.	guest_id
Rooms	Stores room number, type, rate, and status.	room_id
Bookings	Connects a guest to a room for a date range.	booking_id
Payments	Stores payment transactions for a booking.	payment_id
Services	Stores the hotel's additional services and prices.	service_id
Booking_Services	Records which services were used for each booking.	(booking_id, service_id)

3. Data redundancy and anomalies

Data redundancy means storing the same fact repeatedly. For example, if a guest's name, email, and phone number were copied into every booking row, the same guest details would appear many times. If the guest changed their phone number, every copied row would need to be changed.

Anomaly	Example in a poorly designed table	Effect
Insertion anomaly	A new room cannot be stored until a booking is created.	Room details may be missing until the first customer books it.
Update anomaly	A guest's phone number appears in five booking rows.	Changing only some rows creates conflicting phone numbers.
Deletion anomaly	Deleting the only booking for a guest also deletes their contact details.	Useful guest information is lost unintentionally.

4. Functional dependencies

A functional dependency means one attribute, or a group of attributes, determines another attribute. The proposed dependencies are:

```
guest_id -> guest_name, email, phone, address
room_id -> room_number, room_type, price_per_night, status
booking_id -> guest_id, room_id, check_in, check_out, booking_status
payment_id -> booking_id, amount, payment_date, payment_method
service_id -> service_name, service_price
(booking_id, service_id) -> quantity
```

The primary key determines the other non-key attributes in its table. In Booking_Services, the combination of booking_id and service_id identifies one service line for a booking. A guest can have multiple bookings, so guest_id does not determine booking_id.

5. How poor design causes anomalies

If all information were stored in one large table, room prices and guest details would be repeated for every booking and service. Repeated facts increase storage, create inconsistent values, and make it difficult to delete or update information safely. A single table also makes it harder to represent rooms or services that have not yet been booked or used.

6. Improvements and normalization

The design separates information into six related tables. This reduces repetition and follows the main ideas of normalization:

Normal form	How this design follows it
First Normal Form (1NF)	Each field stores one value; services are not kept as comma-separated lists in a booking row.
Second Normal Form (2NF)	Booking_Services uses a composite key; quantity depends on the full pair (booking_id, service_id), not only one part.
Third Normal Form (3NF)	Guest, room, and service descriptions are stored in their own tables instead of being repeated in bookings.

7. Relationship overview

```
Guests (1) -------- (many) Bookings (many) -------- (1) Rooms
                           |
                           | (1 to many)
                           v
                       Payments

Bookings (1) ---- (many) Booking_Services (many) ---- (1) Services
```

8. Conclusion

The proposed design reduces data redundancy and helps prevent insertion, update, and deletion anomalies. Primary keys identify records, foreign keys connect related records, and the Booking_Services bridge table handles the many-to-many relationship between bookings and services.